

ORGCMP L11 – Laboratório de Organização de Computadores

Prof. Jean Marcos Laine
Prof. Bruno da Silva Rodrigues
Prof. Wallace Rodrigues de Santana

Faculdade de Computação e Informática – FCI
Univ. Presbiteriana Mackenzie

*Siga as instruções abaixo e resolva os exercícios solicitados. Depois, elabore um relatório para a atividade usando o template **Relatório de Práticas de Lab Org.docx**, disponível no Moodle, seguindo e respeitando o modelo.*

Lab #2 – Funções e Estruturas de Repetição

Atividade (mín. 2 e máx. três alunos por grupo)

Reflexão Inicial

Saber escrever estruturas de repetição, organizar funções e utilizar desvios e branches em assembly é de extrema importância, mesmo para programadores que geralmente trabalham com linguagens de alto nível. Aqui estão algumas razões pelas quais essa prática é valiosa e como pode melhorar a programação de alto nível:

1. **Melhor Compreensão das Linguagens de Programação:** A programação em assembly oferece uma visão detalhada do funcionamento interno de um computador. Ao escrever estruturas de repetição e funções em assembly, você ganha um entendimento profundo de como essas estruturas são implementadas em uma linguagem de alto nível.
2. **Portabilidade e Abstração:** A programação em assembly pode ensinar como abstrair detalhes de hardware e criar código portátil. Mesmo que você não escreva código assembly para diferentes arquiteturas, essa habilidade ajuda a criar código de alto nível mais independente de plataforma.

O uso de funções na programação é fundamental por várias razões importantes:

1. **Reutilização de Código:** Funções permitem que você encapsule um bloco de código que realiza uma tarefa específica. Essa encapsulação facilita a reutilização desse código em diferentes partes do programa. Em vez de repetir o mesmo código várias vezes, você pode simplesmente chamar a função sempre que precisar executar a tarefa.
2. **Abstração:** Funções ajudam a abstrair complexidade. Em vez de se preocupar com os detalhes internos de como uma tarefa é realizada, você pode simplesmente chamar a função que a realiza. Isso simplifica a escrita e a compreensão do código, tornando-o mais legível e manutenível.
3. **Organização do Código:** O uso de funções ajuda a organizar o código de forma mais lógica e estruturada. Cada função pode ter uma responsabilidade clara e ser nomeada de acordo com

essa responsabilidade, tornando mais fácil para você e outros desenvolvedores entenderem o propósito de cada parte do código.

4. **Facilita a Manutenção:** Quando você precisa fazer alterações em um programa, é muito mais fácil se essas alterações estiverem contidas em funções separadas. Isso minimiza o risco de introduzir erros em outras partes do código e torna a manutenção mais segura e controlada.
5. **Divisão do Trabalho:** Em projetos grandes, várias pessoas podem estar trabalhando simultaneamente em diferentes partes do código. O uso de funções facilita a divisão do trabalho, pois cada desenvolvedor pode se concentrar em implementar e testar funções específicas sem interferir no trabalho dos outros.
6. **Testabilidade:** Funções bem definidas são mais fáceis de testar. Você pode criar testes unitários para cada função individualmente, garantindo que ela funcione conforme o esperado. Isso simplifica a detecção de erros e a garantia de qualidade do software.
7. **Legibilidade:** Código que utiliza funções é geralmente mais legível. Os nomes das funções podem ser descritivos, o que torna o código mais autoexplicativo. Isso facilita a compreensão do código, tanto para você quanto para outros desenvolvedores que possam trabalhar nele.
8. **Melhor Composição:** Você pode criar programas complexos com base em funções mais simples. Isso facilita a criação de sistemas complexos, dividindo-os em partes menores e gerenciáveis.
9. **Melhor Documentação:** Funções bem nomeadas servem como documentação natural do código. Outros desenvolvedores podem entender rapidamente o propósito e o comportamento de uma função apenas olhando seu nome e assinatura.

Em resumo, o uso de funções na programação não apenas facilita o desenvolvimento de software mais organizado e legível, mas também melhora a reutilização de código, a manutenção, a colaboração e a qualidade geral do software. É uma prática fundamental que contribui significativamente para a eficiência e a qualidade do desenvolvimento de software.

ORIENTAÇÕES INICIAIS

1. Abra o simulador MARS para editar, montar (gerar o código binário) e testar o(s) programa(s) que será(ão) desenvolvido(s).
2. Lembrando, a montagem do programa pode ser feita em Run -> Assemble ou clicando no botão de atalho .
3. Em um primeiro momento, execute seu código no modo passo-a-passo, clicando em  e observando o conteúdo dos registradores, das memórias, as instruções que estão sendo executadas etc. Analise se sua lógica está adequada ao exercício proposto.
4. Depois, faça a execução normal para realizar seus casos de teste e validar seu programa, clicando em .

Exercício 1 (3,0 pontos) – Neste exercício, você irá criar um programa em assembly MIPS que calcula o fatorial de um número inteiro não negativo usando função (a chamada da função é através da instrução JAL e o retorno com JR). O programa deve solicitar ao usuário que insira um número inteiro e, em seguida, chamar a função para calcular o fatorial desse número, depois imprimir o resultado na tela.

Valide a entrada do usuário para não aceitar como entrada um inteiro negativo!

Tarefas:

- Implemente uma função chamada fatorial que calcula o fatorial de um número. Lembre-se: ao implementar uma função seu código deve fazer uso do JAL e do JR para chamar a função e retornar dela, respectivamente.
- A função fatorial deve receber um argumento inteiro (o número para o qual calcular o fatorial) através de um registrador e armazenar o resultado também em um registrador.
- No programa principal, solicite ao usuário que insira um número inteiro não negativo (faça a validação).
- Chame a função fatorial com o número digitado pelo usuário como argumento. O argumento para a função deve ser armazenado no registrador \$a0.
- Imprima o resultado na tela.

Pontuações:

- (0,25 pto) Ler a entrada do usuário;
- (0,5 pto) Validar a entrada do usuário. Se o número for menor ou igual a zero, solicitar que digite novamente enquanto a entrada for inválida;
- (0,25 pto) Criar e chamar a função fatorial corretamente;
- (1,75 pto) Calcular o fatorial e imprimir na tela o resultado;
- (0,25 pto) Comentar o código para explicar as etapas essenciais e o funcionamento do seu programa.

Exercício 2 (3,0 pontos) – A sequência numérica de Fibonacci é um conceito clássico em Ciência da Computação. Muitos padrões na natureza foram observados para seguir a sequência de Fibonacci. Alan Turing, um homem que é considerado o pai da Ciência da Computação, dedicou a última parte de sua vida à pesquisa em biologia e física que envolveu a Sequência de Fibonacci.

- Anteriormente definido, o n -ésimo número de Fibonacci é:

$$F_n = F_{n-1} + F_{n-2}$$

Sendo, $F_1=1$ e $F_0=0$

- Um exemplo de sequência é 0, 1, 1, 2, 3, 5, 8, 13, 21, 34... Como você pode ver, você soma os últimos dois números para obter o próximo número na sequência.

Escreva um programa usando a linguagem de montagem do MIPS para gerar a sequência de Fibonacci através de uma função (a chamada da função é através da instrução JAL e o retorno com JR) e imprimir a sequência com o separador vírgula entre os números, conforme exemplo acima. O usuário deve informar, via teclado, a quantidade de termos da sequência.

Seu código deve estar organizado da seguinte forma:

No “Main”:

- O main deve solicitar ao usuário a quantidade de termos da série;
- Chamar uma função *Fibonacci* para gerar a sequência e imprimi-la na tela, igual o exemplo dado (uma única linha com os valores separados por vírgula). Lembre-se: ao implementar uma função seu código deve fazer uso do JAL e do JR para chamar a função e retornar dela, respectivamente.

Pontuações:

- (0,25 pts) Ler a entrada do usuário;
- (0,5 pts) Validar a entrada do usuário. Se o número for menor ou igual a zero, solicitar que digite novamente enquanto a entrada for inválida;
- (0,25 pts) Criar e chamar a função fatorial corretamente;
- (1,75 pts) Gerar e imprimir na tela a sequência de Fibonacci formatada;
- (0,25 pts) Comentar o código para explicar as etapas essenciais e o funcionamento do seu programa.

Exercício 3 (4,0 pontos) – Neste exercício, você irá criar um programa em assembly MIPS que converte um número decimal em sua representação binária usando funções. O programa deve solicitar ao usuário que insira um número decimal e, em seguida, chamar uma função para realizar a conversão para binário. Utilize o método das divisões sucessivas para a conversão.

O resultado binário deve ser impresso na tela.

Tarefas:

1. Implemente uma função chamada *decimal_para_binario* que converte um número decimal em sua representação binária. Lembre-se: ao implementar uma função seu código deve fazer uso do JAL e do JR para chamar a função e retornar dela, respectivamente.
2. A função *decimal_para_binario* deve receber o número decimal como argumento e converter o número para a representação binária.
3. No programa principal, solicite ao usuário que insira um número decimal inteiro positivo.
4. Chame a função *decimal_para_binario* para realizar a conversão do número digitado pelo usuário.
5. Imprima o resultado binário na tela. **Atenção**: o resultado deve ter a quantidade exata de bits para representar o número. Exemplo de conversão do decimal 10 => 1010
6. Lide com casos de erro, como números negativos ou entradas inválidas.

Pontuações:

- (0,25 pts) Ler a entrada do usuário;
- (0,5 pts) Validar a entrada do usuário. Se o número for menor ou igual a zero, solicitar que digite novamente enquanto a entrada for inválida;
- (0,25 pts) Criar e chamar a função fatorial corretamente;
- (2,75 pts) Converter um número decimal em sua representação binária e imprimir na tela o resultado;
- (0,25 pts) Comentar o código para explicar as etapas essenciais e o funcionamento do seu programa.

Dica: você pode alocar um espaço em memória e ir armazenando o resto das divisões. Depois, é só percorrer esse espaço e imprimir os valores armazenados (número binário). É possível fazer isso usando as instruções sb (store byte) e lb (load byte). A seguir, deixo um exemplo de código mostrando como fazer isso.

```
.data
binario: .space 32

.text
li $t0,5
li $t1,2

divu $t0, $t1
mfhi $s0 #RESTO DA DIVISAO
mflo $s1 #QUOCIENTE

move $t2, $zero
sb $s0, binario($t2)

divu $s0, $t1
mfhi $s0 #RESTO DA DIVISAO
mflo $s1 #QUOCIENTE

addi $t2,$t2,1
sb $s1, binario($t2)

move $t2, $zero
lb $a0,binario($t2)
li $v0, 1
syscall

addi $t2,$t2,1
lb $a0,binario($t2)
li $v0, 1
syscall
```

Critérios de avaliação

A nota dessa atividade é calculada de acordo com o correto funcionamento ou não do que foi solicitado no exercício e as pontuações estabelecidas. A tabela a seguir contém critérios de avaliação que podem **reduzir** a nota final da atividade.

ITEM INDESEJÁVEL	REDUÇÃO DE NOTA
Não entregou o relatório	-5,0
O relatório não demonstra todos os casos de testes no exercício	-1,0 em cada exercício
O código (.asm) do exercício não compila	Zerado
Não há identificação do grupo (no código-fonte e no relatório).	-1,0
Arquivo fonte (.asm) do exercício não enviado	Zerado
Código não atende ao que foi solicitado	Zerado
Código não contém comentários	-1,0
A atividade não foi apresentada pelo grupo	Atividade fica sem Nota

Tabela - Critérios de avaliação (redução de nota).

Observações:

- Esta atividade pode ser feita em grupo, conforme definido pelo professor;
- Não serão aceitas entregas fora do prazo;
- Organize e comente seu código fonte explicando o que faz os principais blocos/trechos programados;
- Teste e valide seu programa para todos os casos que validam as funcionalidades solicitadas no exercício, antes de enviar no Moodle;

O que entregar?

- **Entregar** o relatório solicitado no formato PDF de acordo com o template disponibilizado;
- **Entregar** os códigos fontes escritos em MIPS. Nomeie os arquivos de acordo com o exercício, exemplo: ex1.asm, ex2.asm etc. Coloque no cabeçalho o nome dos integrantes;
- **Atenção:** Apenas um aluno do grupo precisa submeter a atividade no Moodle.

O que será avaliado?

- Corretude do programa (deve estar “compilando sem erros” e realizando o que foi solicitado);
- O quão fiel é o programa quanto à descrição do enunciado;
- Indentação, comentários, organização e legibilidade do código;
- Qualidade e completude do relatório elaborado. Observação: os *prints* de teste inseridos no relatório precisam mostrar todos os testes que confirmam que o programa atende às diferentes solicitações do exercício.

Referência

HENNESSY, J. L.; PATTERSON, D. A.; LARUS, J. R.; MACHADO FILHO, N. Organização e projeto de computadores: a interface hardware/software. 5ª Edição, Rio de Janeiro: Elsevier, 2014.

Disponível em:

- **MINHA BIBLIOTECA - BIBLIOTECA DIGITAL** - oferece mais de 13.310 títulos e é formada pelo consórcio das **quatro principais editoras** de livros acadêmicos do Brasil: Grupo Gen - Atlas, Grupo A, Saraiva e Manole. Essas editoras se uniram para oferecer às instituições de ensino superior uma plataforma prática e inovadora para acesso a um conteúdo científico e técnico de qualidade. Ainda oferece cerca de 1.000 obras das editoras convidadas: Cengage, Cortez, Grupo Autêntica e Zahar. Cada usuário possui uma conta individual no sistema, preservando suas marcações e anotações. Dispõe de link que referencia suas citações diretas e permite imprimir parte do conteúdo.

Clique [aqui](#) para acessar.

<https://www.mackenzie.br/biblioteca/recursos-de-pesquisa/livros-eletronicos>

